



Intelligence Artificielle - IA

Séance 02 - Le neurone artificiel



Dr. Mohamed Ould Djibril

November 12, 2023

Département d'Informatique et Méthodes Quantitatives
Institut Supérieur de Comptabilité et d'Administration
des Entreprises - ISCAE

mohamed.djibril@gmail.com

Sommaire

1 Le neurone artificiel

Le neurone formel

Définition et exemples

Limitations

Le perceptron

Définition et exemples

Architecture et mise en équation

Limitations

Principe d'apprentissage d'un neurone

Algorithme d'apprentissage par correction d'erreurs

Algorithme de la descente du gradient classique

Algorithme de la descente du gradient stochastique

Les fonctions d'erreur

2 Conclusion

Introduction

Neurone artificiel

- L'unité de base de calcul d'un réseau de neurones est appelée **Neurone Artificiel**;
- Inspirée de: neurones biologiques = cellules neuronales = unités de traitement neuronales;
- Son principe général est de retourner une information en sortie à partir de plusieurs informations en entrée.

Introduction

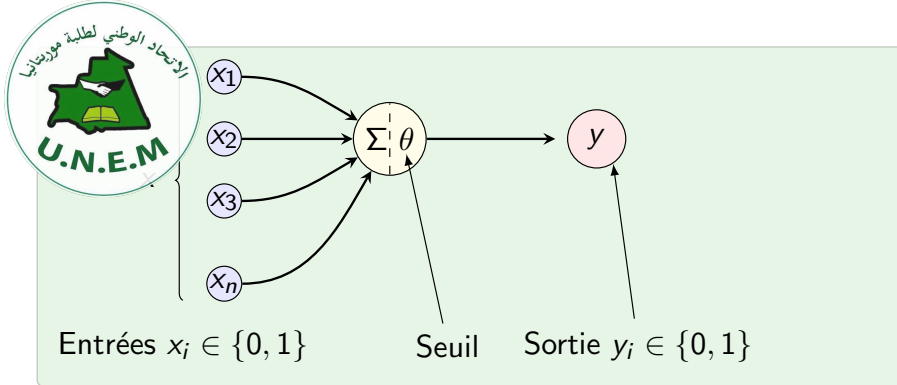
Neurone artificiel

- L'unité de base d'un réseau de neurones est appelée **Neurone Artificiel**
- Inspirée de: neurones biologiques = cellules neuronales = unités de traitement neuronales

Evolution des neurones artificiels

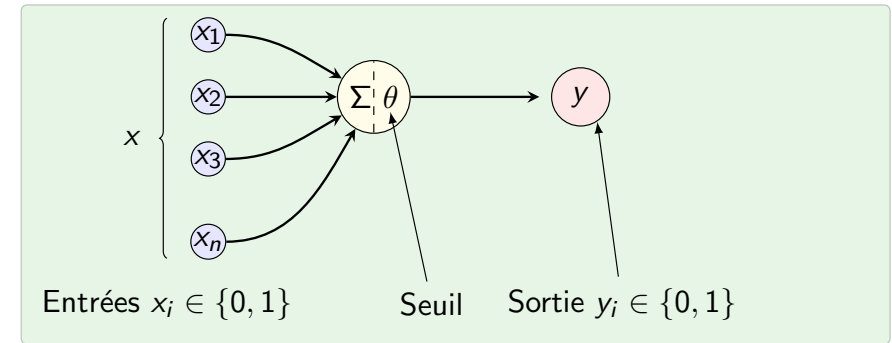
- Le neurone formel - **McCulloch** et **Walter Pitts**, 1943;
- Le perceptron - **Frank Rosenblatt** en 1958;
- Le neurone artificiel - ou **Le neurone sigmoïde**.

Neurone artificiel de McCulloch



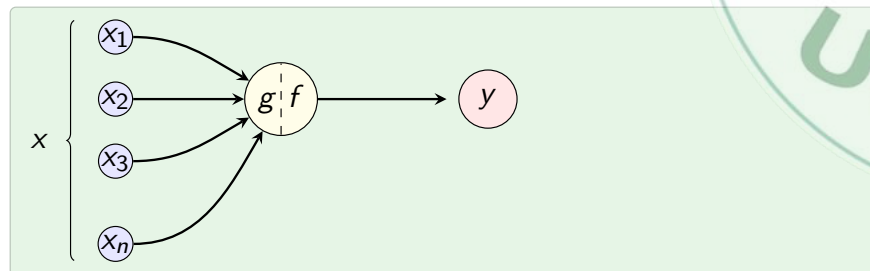
- Le premier modèle, le plus simple, **noté neurone MP** McCulloch et Pitts
- Les entrées x_i sont **binaires**;
- La sortie y est aussi binaire, $y = 1$ c'est à dire **neurone active**.

Neurone artificiel de McCulloch



- Σ est la somme des entrées;
- Si la somme des entrées est supérieure à θ alors on active le neurone $y = 1$, sinon $y = 0$ est le neurone est dit non active;
- θ est appelé **seuil d'activation**.

Neurone artificiel de McCulloch



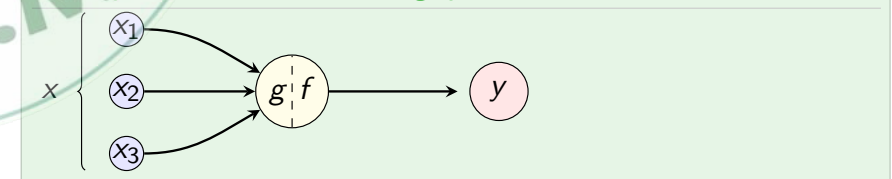
Pour simplifier, introduisons les deux fonctions g et f définies :

$$g(x_1, x_2, \dots, x_n) = g(\mathbf{x}) = \sum_{i=1}^n x_i$$

$$y = f(g(\mathbf{x})) = \begin{cases} 1 & \text{si } g(\mathbf{x}) \geq \theta \\ 0 & \text{si } g(\mathbf{x}) < \theta \end{cases}$$

Neurone artificiel de McCulloch

Exemple 1 : la fonction logique OU



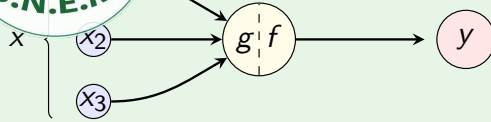
- Le neurone est active si au moins l'une des entrées vaut 1;
- il suffit de fixer $\theta = 1$:

$$y = f(g(\mathbf{x})) = \begin{cases} 1 & \text{si } g(\mathbf{x}) \geq 1 \\ 0 & \text{si } g(\mathbf{x}) < 1 \end{cases}$$

Neurone artificiel de McCulloch



le 2 : la fonction logique ET

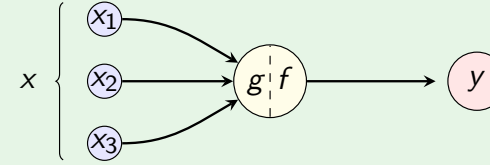


- Le neurone est active si $x_i = 1$ pour tous les i ;
- il suffit de fixer $\theta = 3$:

$$y = f(g(\mathbf{x})) = \begin{cases} 1 & \text{si } g(\mathbf{x}) \geq 3 \\ 0 & \text{si } g(\mathbf{x}) < 3 \end{cases}$$

Neurone artificiel de McCulloch

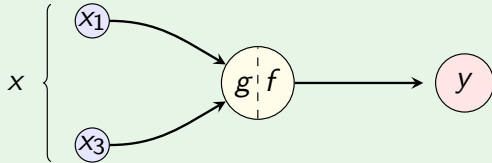
Exercice



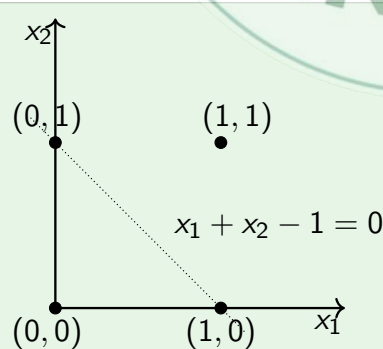
- Définir le neurone MP pour calculer la fonction NON
- penser à définir pour chaque fonction le nombre de variables en entrée
- ensuite fixer θ :

Interprétation géométrique

Fonction **OU**, deux entrées x_1 et x_2 , $\theta = 1$

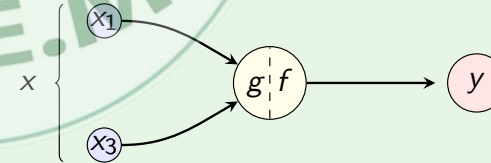


- Un neurone MP de deux entrées binaires x_1 et x_2 divise les 4 points en deux classes;
- Les combinaisons qui vont produire une sortie $y = 0$ et ceux qui produiront une sortie $y = 1$;
- Ces deux classes sont séparées par la ligne définie par l'équation: $\sum_{i=1}^n x_i - \theta = 0$.

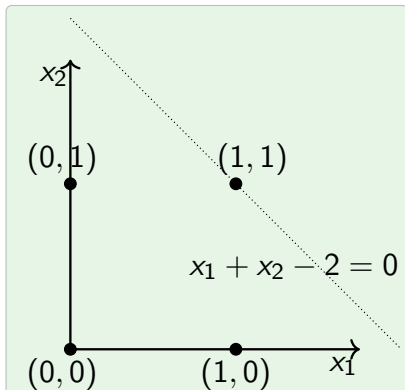


Interprétation géométrique

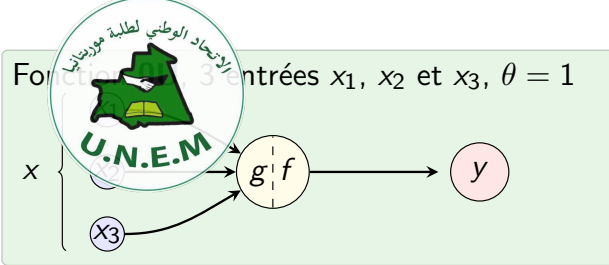
Fonction **ET**, deux entrées x_1 et x_2 , $\theta = 2$



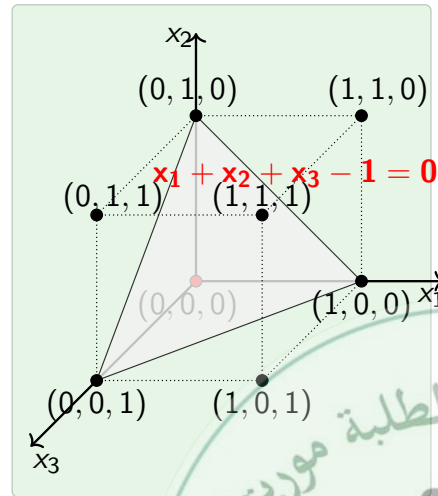
- Un neurone MP de deux entrées binaires x_1 et x_2 divise les 4 points en deux classes;
- Les combinaisons qui vont produire une sortie $y = 0$ et ceux qui produiront une sortie $y = 1$;
- Ces deux classes sont séparées par la ligne définie par l'équation: $\sum_{i=1}^n x_i - \theta = 0$.



Interprétation géométrique



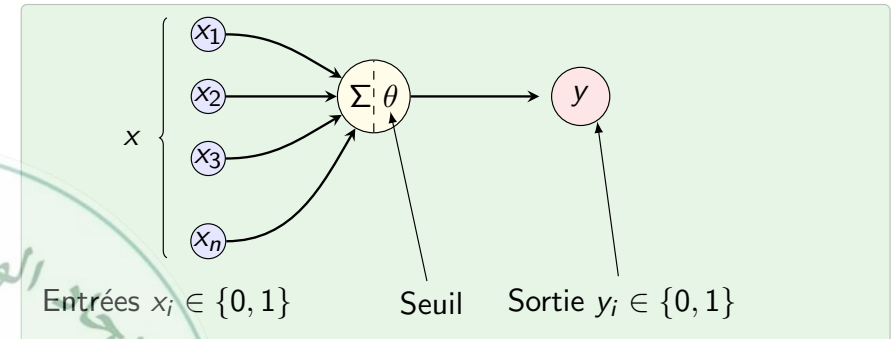
- Un neurone MP de 3 entrées binaires x_1 , x_2 et x_3 divise les 8 points en deux classes;
- Les combinaisons qui vont produire une sortie $y = 0$ et ceux qui produiront une sortie $y = 1$;
- Ces deux classes sont séparées par le plan défini par l'équation: $\sum_{i=1}^n x_i - \theta = 0$.



Neurone artificiel de McCulloch

Conclusion

- Un seul neurone MP peut être utilisé pour représenter les fonctions booléennes qui sont linéairement séparables.



Neurone artificiel de McCulloch

Limitations:

- Un neurone MP ne peut pas représenter les fonctions booléennes non linéairement séparables, exemple XOR;
- Les entrées ne sont que des variables booléennes, et les réels?
- Le paramètre θ est fixé manuellement;
- Pas de moyen pour favoriser certaines entrées qui peuvent être plus importantes que les autres.

Le modèle suivant: — > le perceptron.

Le perceptron

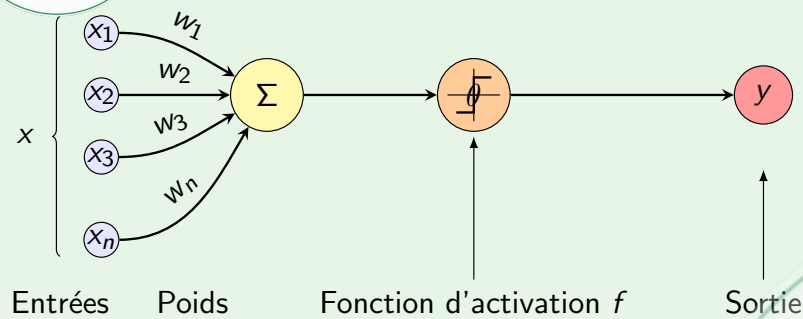
Introduction

- Modèle introduit en 1958 par Rosenblatt et amélioré par Minsky et Papert en 1969;
- Il s'agit d'un modèle de calcul inspiré du modèle de Warren McCulloch et Walter Pitts;
- Il introduit des poids pour les entrées du modèle et un mécanisme d'apprentissage de ces poids;
- C'est le premier système artificiel capable d'apprendre par expérience;
- Les entrées ne sont plus limitées uniquement aux valeurs booléennes;
- Il est aussi appelé **perceptron linéaire à seuil**.

Le perceptron



Représentation graphique



Le perceptron

Définition d'un perceptron

Un neurone de type **perceptron** est composé de :

- **Les entrées** x_i : n valeurs $x_1, \dots, x_n$
- **Les poids** w_i : n valeurs $w_1, \dots, w_n$. w_i désigne le poids à appliquer à l'entrée x_i lors du calcul de la sortie du neurone.
- **Le biais** θ
- **La fonction d'activation** f
- **La sortie** y

Les x_i peuvent être des réels, les w_i des entiers ou des réels.

17 / 78

18 / 78

Le perceptron

Définition d'une fonction d'activation

- soit $x = (x_1, \dots, x_n)$ le vecteur des entrées.
- et soit la fonction g :

$$g(x_1, x_2, \dots, x_n) = g(\mathbf{x}) = \sum_{i=1}^n w_i x_i - \theta$$

la quantité $g(\mathbf{x})$ est appelée **potentiel du neurone**.

- La sortie y du neurone est calculée par : $y = f(g(\mathbf{x}))$
- La sortie y du neurone est calculée par l'application de la fonction f à son potentiel.
- La fonction f est appelée **fonction d'activation** ou **fonction de transfert**.

Le perceptron

La fonction d'activation du perceptron

- La fonction d'activation caractérise un neurone.
- Il existe plusieurs types de fonctions d'activation.
- Le perceptron utilise la fonction **Heaviside** ou la fonction seuil:

$$f(t) = \begin{cases} 1 & \text{si } t \geq 0 \\ 0 & \text{sinon} \end{cases} \quad \text{ou} \quad f(t) = \begin{cases} 1 & \text{si } t \geq 0 \\ -1 & \text{sinon} \end{cases}$$

- Un neurone utilisant cette fonction est appelé perceptron linéaire à seuil.

```
def fonctionSeuil(t):
    if t >= 0:
        return 1
    return 0
```

19 / 78

20 / 78

Le perceptron



Architecture et mise en équation

Le perceptron linéaire à seuil prend en entrée n valeurs $x_1, \dots, x_n$ et calcule une sortie y .

- Il est défini par $n + 1$ constantes :
 - Les coefficients ou poids synaptiques $w_1, \dots, w_n$ et
 - Le seuil θ appelé aussi **biai**.
- La sortie y est calculée par :

$$g(x_1, x_2, \dots, x_n) = g(\mathbf{x}) = \sum_{i=1}^n w_i x_i$$

$$y = f(g(\mathbf{x})) = \begin{cases} 1 & \text{si } g(\mathbf{x}) \geq \theta \\ 0 & \text{sinon} \end{cases}$$

Le perceptron

Architecture et mise en équation

- Pour simplifier la notation, nous introduisons une entrée supplémentaire $x_0 = 1$ et on lui associe un coefficient $w_0 = -\theta$. On a donc :

$$\sum_{i=1}^n w_i x_i \geq \theta \iff \sum_{i=0}^n w_i x_i \geq 0$$

$$g(\mathbf{x}) = w_0 + \sum_{i=1}^n w_i x_i = \sum_{i=0}^n w_i x_i$$

$$\text{donc : } y = f(g(\mathbf{x})) = \begin{cases} 1 & \text{si } g(\mathbf{x}) \geq 0 \\ 0 & \text{sinon} \end{cases}$$

- w_0 est donc appelé le **biai**

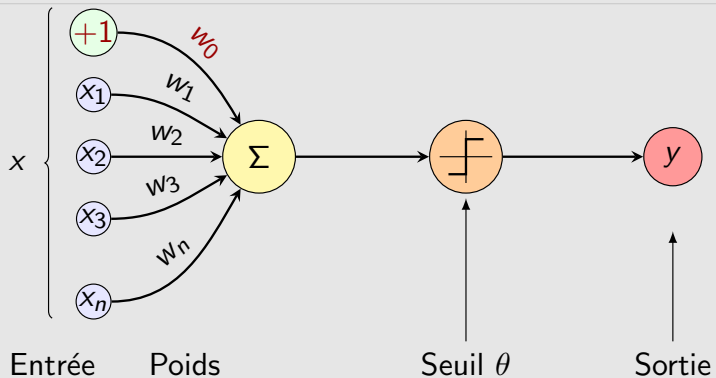
21 / 78

22 / 78

Le perceptron

Le perceptron

Architecture et mise en équation



```
def prediction(X, W, b):
    return fonctionSeuil(np.matmul(X,W)+b)
```

Exemple :

- Soit $x = (x_1, x_2) \in \mathbb{R}^2$ le vecteur d'entrées
- et soit $w = (-1, 1, 2)$ le vecteur des poids;
- le seuil $-\theta = w_0 = -1$ et $g(x) = x_1 + 2x_2 - 1$
- la sortie :

$$y = f(g(\mathbf{x})) = \begin{cases} 1 & \text{si } x_1 + 2x_2 - 1 \geq 0 \\ 0 & \text{sinon} \end{cases}$$

- Par exemple, $f(0, 0) = 0$, $f(1, 1) = 1$ et $f(1, 0.5) = 1$
- Comme pour le neurone de MP, on voit là aussi que les deux classes sont séparées par la droite définie par l'équation $x_1 + 2x_2 - 1 = 0$

Le perceptron



Implémentation

```
X = np.array([0.5,0.5])
W = np.array([1., 2.])
b = -1.

y = prediction(X, W, b)

print('Entrées (x1,x2) = ',X)
print('La sortie y = ',y)
```

25 / 78

Le perceptron

Architecture et mise en équation

- points vérifiant $\theta + \langle w, x \rangle = 0$ hyperplan défini par θ et w ;
- points vérifiant $\theta + \langle w, x \rangle > 0$ points d'un côté de l'hyperplan;
- points vérifiant $\theta + \langle w, x \rangle < 0$ points de l'autre côté de l'hyperplan.

Théorème

Un perceptron linéaire à seuil à d entrées divise l'espace des entrées $\mathbb{R}$ en deux sous-espaces délimités par un hyperplan.

Exercice 01:

Démontrer que le XOR ne peut pas être calculé par un perceptron linéaire à seuil.

Le perceptron

Architecture et mise en équation : généralisation

- Soit $x = (x_1, \dots, x_d) \in \mathbb{R}^d$ le vecteur d'entrées
- et soit $w = (w_0, \dots, w_d)$ le vecteur des poids;
- $\langle w, x \rangle$ le produit scalaire entre w et x
- le seuil $-\theta = w_0$ et $\langle w, x \rangle = \sum_{i=1}^d w_i x_i$
- $\langle w, x \rangle + w_0 = 0$ est l'équation d'un hyperplan qui sépare x en deux demi-espaces correspondant aux deux classes.
- Par exemple, si $d = 2$ on retrouve l'équation d'une droite :
 $\langle w, x \rangle + w_0 = w_1 x_1 + w_2 x_2 + w_0 = 0$ donne
 $x_2 = -w_1/w_2 x_1 - w_0/w_2$

26 / 78

Architecture et mise en équation

Exercice 02

Soit un perceptron dont le vecteur $w^T = (2, 1, 1)^T$.

- 1 Tracer sur un diagramme le séparateur linéaire obtenu par ce perceptron et hachurer la surface correspondante à la partie du plan où le perceptron retourne la valeur 1.
- 2 Lesquels parmi les perceptrons caractérisés par les vecteurs de pondération suivants ont le même hyperplan et qui retourne exactement le même résultat de classification que le perceptron donné dans 1. ?

(a) $(w_0, w_1, w_2)^T = (1, 0.5, 0.5)^T$

(b) $(w_0, w_1, w_2)^T = (200, 100, 100)^T$

(c) $(w_0, w_1, w_2)^T = (\sqrt{2}, \sqrt{1}, \sqrt{1})^T$

(d) $(w_0, w_1, w_2)^T = (-2, -1, -1)^T$.

(Source : le site de François DENIS - <http://pageperso.lif.univ-mrs.fr/francois.denis/>)



les paramètres w_i avec l'exemple OU logique

x_1	x_2	OU	
0	0	0	$w_0 + \sum_{i=1}^2 w_i x_i < 0$
1	0	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$
0	1	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$
1	1	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$

$$w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 \implies w_0 < 0$$

$$w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 \implies w_2 \geq -w_0$$

$$w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 \implies w_1 \geq -w_0$$

$$w_0 + w_1 \cdot 1 + w_2 \cdot 1 \geq 0 \implies w_1 + w_2 \geq -w_0$$

Une solution: $w_0 = -0.5, w_1 = 1, w_2 = 1$

Analysons les paramètres w_i

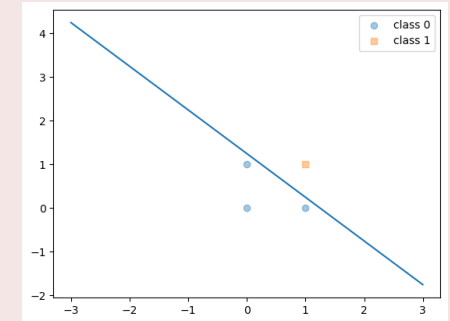
```
d = 4
X0 = np.array([0, 0, 1, 1])
X1 = np.array([0, 1, 0, 1])
X = np.vstack([X0, X1])
ytrain = np.array([0, 0, 0, 1])

W = np.array([0.8, 0.8])
b = -1.

y = prediction(X[:,3], W, b)
print('(x1,x2) = ',X[:,3])
print('(w1,w2) = ',W)
print('biai = ',b)
print('La sortie y = ',y)
```

Exercice:

Pour $(w_1, w_2) = [0.8 \ 0.8]$ notre perceptron calcule bien OU. Modifier les paramètres et observer.



29 / 78

30 / 78

Position du problème

- **Problème** : Trouver les poids d'un perceptron qui classe correctement un ensemble S d'apprentissage de $\mathbb{R}^n \times \{0, 1\}$.
- $D = \{(x^1, y^1), \dots, (x^n, y^n)\}; x^i \in \mathbb{R}^d$ et $y^i \in \{0, 1\}$
- Si $y^i = 1$ on dit que x^i est un exemple positif, si $y^i = 0$ on dit que x^i est un exemple négatif.
- D est appelé l'ensemble d'entraînement ou d'apprentissage ou le corpus d'apprentissage.
- **Objectif** : Générer un perceptron qui retourne 1 pour tous les exemples positifs et 0 pour tous les exemples négatifs.

Deux types d'erreurs

- ❶ Erreur de classification de l'exemple x^i négatif comme étant positif ($y^i = 1$) et $w_0 + \langle w, x \rangle > 0$
- ❷ Erreur de classification de l'exemple x^i positif comme étant négatif ($y^i = 0$) et $w_0 + \langle w, x \rangle < 0$

```
def backward(x, W, b):
    y_predicted = prediction(x, W, b)
    erreur = y - y_predicted
    return erreur
```




cas négatif et $w_0 + \langle w, x \rangle > 0$

$$w_0 + \sum_{i=1}^d w_i x_i$$

diminuer w_0

- Si $x_i > 0$ diminuer w_i
- Si $x_i < 0$ augmenter w_i

2^{eme} Cas: x^i positif et $w_0 + \langle w, x \rangle < 0$

$$\text{Augmenter } w_0 + \sum_{i=1}^d w_i x_i$$

- augmenter w_0
- Si $x_i > 0$ augmenter w_i
- Si $x_i < 0$ diminuer w_i

33 / 78

Rosenblatt's Perceptron Training Rule

- Entrées:** Un ensemble d'entraînement D contenant n exemples (x_i, y_i) ; $x_i \in \mathbb{R}^d$ et $y_i \in \{0, 1\}$
- Initialiser les poids w et le seuil b à 0 ou à de petites valeurs aléatoires.;
- répéter**
 - for chaque exemple i : $(x_i, y_i) \in S$ do**
 - Calculer la sortie $\hat{y}_i$ pour l'entrée x_i ;
 - $w = w + \eta \cdot (y_i - \hat{y}_i) x_i$;
 - $b = b + \eta \cdot (y_i - \hat{y}_i)$;
- jusqu'à toutes les entrées sont classées correctem.;**
- Sortie:** Le perceptron défini par les paramètres (w, b) ;

—> Algorithme d'apprentissage par correction d'erreur

- A chaque fois que l'on présente un nouvel i ième exemple, on ajuste les poids selon que le perceptron l'a correctement classé ou non.

$$w_j \leftarrow w_j + \Delta w_j$$

où

$$\Delta w_j = \eta (y^i - \hat{y}^i) x_j^i$$

$$w_j = w_j + \eta \cdot (y^i - \hat{y}^i) x_j^i$$

η est appelé le **taux d'apprentissage** (learning rate)

- L'algorithme s'arrête lorsque tous les exemples ont été présentés sans modification d'aucun poids.

34 / 78

```
def train(self, X, y):
    n = len(X)
    d = X.shape[1]
    weights = np.zeros((d))
    bias = 0
    for epoch in range(epochs):
        for i in range(n):
            y_pred = predict(X[i])
            weights = weights + lr * (y[i] - y_pred) * X[i]
            bias = bias + lr * (y[i] - y_pred)
    return weights, bias
```



En phase de calcul, les constantes du perceptron sont les poids synaptiques alors que les variables sont les entrées.

- En phase d'apprentissage, les poids sont variables alors que les entrées du perceptron sont des constantes.

Analyse de l'algorithme d'apprentissage

- Le choix d'un élément de D est réalisé aléatoirement ou bien suivant un ordre précis?
- Quel est le critère d'arrêt de la boucle principale de l'algorithme?
- Doit-on arrêter la boucle Lorsque les poids ne sont plus modifiés pendant un certain nombre d'étapes ?

37 / 78

Analyse de l'algorithme d'apprentissage

- L'algorithme d'apprentissage du perceptron est une procédure de correction d'erreur entre y et $\hat{y}$.
- Les poids ne sont pas modifiés lorsque la sortie attendue y est égale à la sortie calculée $\hat{y}$ par le perceptron courant.
- si $\hat{y} = 0$ et $y = 1$, cela signifie que le perceptron n'a pas assez pris en compte les entrées actives (les neurones actifs de l'entrée); $\mathbf{w} = \mathbf{w} + \mathbf{x}_i$ ajoute la valeur de la rétine aux poids synaptiques (renforcement).
- si $\hat{y} = 1$ et $y = 0$, alors $\mathbf{w} = \mathbf{w} - \mathbf{x}_i$; l'algorithme retranche la valeur de la rétine aux poids synaptiques (inhibition).

38 / 78

Analyse de l'algorithme d'apprentissage

- L'algorithme d'apprentissage du perceptron est correct : lorsqu'il converge, l'hyperplan résultat sépare bien les données fournies en entrée.
- L'algorithme est complet : si l'ensemble d'apprentissage D est linéairement séparable, l'algorithme converge.
- Sa complexité est exponentielle, en pratique, on se limite souvent à un nombre d'itérations fixé par avance.

Vocabulaire :

- **Apprentissage en ligne** : ou (online learning en anglais), en parcourant l'ensemble d'apprentissage, l'algorithme réalise l'étape de mise-à-jour des paramètres **pour chaque exemple**.
- **Apprentissage par lot** : ou hors ligne ou (batch learning en anglais), met à jour les paramètres uniquement après avoir **lu tous les exemples** de l'ensemble d'apprentissage ; en d'autres termes, il minimise directement le nombre total d'erreurs.
- **Apprentissage mini-lot** : ou (mini-batch learning en anglais), divise les exemples de données d'apprentissage en paquets de taille approximativement égale, puis met à jour les paramètres après lecture d'un paquet.



lieu de chercher à obtenir un perceptron qui classe correctement tous les exemples, l'approche par descente de gradient va calculer une erreur et cherchera à minimiser cette erreur.

- Nous définissons une fonction de coût $J(w)$ que nous pouvons minimiser afin de mettre à jour nos poids.
- Dans le cas de la fonction d'activation linéaire, nous pouvons définir la fonction de coût $J(w)$ comme la somme des carrés des erreurs; appelée **erreur quadratique**.

$$J(w) = \frac{1}{2} \sum_i (y^i - \hat{y}^i)^2$$

La fraction $\frac{1}{2}$ est juste utilisé pour plus de commodité pour dériver le gradient.

41 / 78

Descente du gradient - une variable

Considérons la fonction $f : \mathbb{R} \rightarrow \mathbb{R}$ définie par : $f(a) = a^2 + 1$.

- Il s'agit de trouver la valeur en laquelle f atteint son minimum, c'est clairement $a_{\min} = 0$ pour lequel $f(a_{\min}) = 1$.
- Retrouvons ceci par la descente de gradient.
- Partant d'une valeur a_0 quelconque, la formule de récurrence est:

$$a_{k+1} = a_k - \eta \nabla f(a)$$

où η est le **pas**, choisi assez petit, et $\nabla f(a) = f'(a) = 2a$.

- Autrement dit :

$$a_{k+1} = a_k - 2\eta a_k.$$

Approche

- Afin de minimiser la fonction de coût $J(w)$ nous utiliserons la descente de gradient
- L'objectif de la méthode de descente de gradient est de trouver un minimum d'une fonction de plusieurs variables le plus rapidement possible.
- L'idée est très simple, on sait que le vecteur opposé au gradient indique une direction vers des plus petites valeurs de la fonction, il suffit donc de suivre d'un pas cette direction et de recommencer.
- Cependant, afin d'être encore plus rapide, il est possible d'ajouter plusieurs paramètres qui demandent pas mal d'ingénierie pour être bien choisis.

42 / 78

```
def f(x):
    return x**2+1

# gradient = dérivée (un vecteur de taille 1)
def grad_f(x):
    return np.array([2*x])
```

```
def gradient_descent(grad, start, learn_rate, n_iter):
    vector = start
    for i in range(n_iter):
        diff = -learn_rate * grad(vector)
        vector += diff
    return vector
```

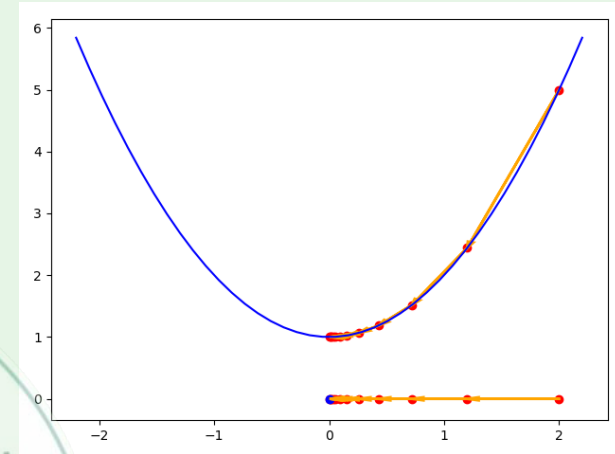


Descente du gradient - une variable

Tableau des valeurs pour un pas $\eta = 0.2$ et une valeur $a_0 = 2$.

k	a_k	$f'(a_k) = \nabla f(a_k)$	$f(a_k)$
0	2	4	5
1	1.2	2.4	2.44
2	0.72	1.44	1.5184
3	0.43	0.86	1.1866
4	0.25	0.5184	1.0671
5	0.15	0.31	1.0241
6	0.093	0.186	1.0087
7	0.055	0.111	1.0031
8	0.033	0.067	1.0011
9	0.020	0.040	1.0004
10	0.012	0.024	1.0001

Descente du gradient - une variable

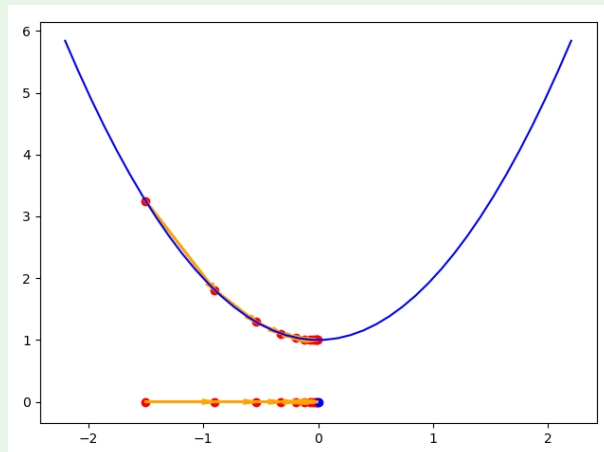


$\eta = 0.2 \quad a_0 = 2$

45 / 78

46 / 78

Descente du gradient - une variable



$\eta = 0.2 \quad a_0 = -1.5$

Descente du gradient - deux variables

Considérons l'exemple de $f(a, b) = a^2 + 3b^2$ dont le minimum est atteint en $(0, 0)$ et appliquons la méthode du gradient.

- Le gradient est donné par la formule :
 $\nabla f(a, b) = \left(\frac{\partial f}{\partial a}(a, b), \frac{\partial f}{\partial b}(a, b) \right) = (2a, 6b)$.
- Tout d'abord, on part d'un point $(a_0, b_0) = (2, 1)$ par exemple.
- On calcule $\nabla f(a_0, b_0) = (4, 6)$. On fixe le facteur $\eta = 0.2$. On se déplace dans la direction opposée à ce gradient :
 $(a_1, b_1) = (a_0, b_0) - \eta \nabla f(a_0, b_0)$
 $= (2, 1) - 0.2(4, 6) = (2, 1) - (0.8, 1.2) = (1.2, -0.2)$.
- On recommence ensuite depuis (a_1, b_1) .



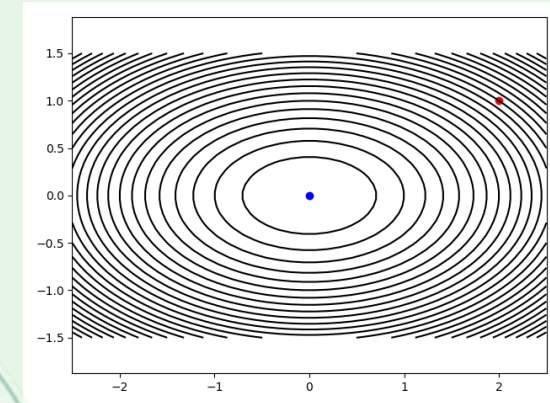
Descente du gradient - deux variables

À chaque étape les valeurs de f tendent vers la valeur minimale, dans notre cas, la suite converge vers $(0, 0)$

k	(a_k, b_k)	$\Delta f(a_k, b_k)$	$f(a_k, b_k)$
0	(2, 1)	(4, 6)	7
1	(1.2, -0.2)	(2.4, -1.20)	1.56
2	(0.72, 0.04)	(1.44, 0.24)	0.523
3	(0.432, -0.008)	(0.864, -0.048)	0.186
4	(0.2592, 0.0016)	(0.5184, 0.0096)	0.067
5	(0.15552, -0.00032)	(0.31104, -0.00192)	0.024
...			
10	$(0.012, 1.02 \cdot 10^{-7})$	$(0.024, 6.14 \cdot 10^{-7})$	0.00014
...			
20	$(7.31 \cdot 10^{-5}, 1.04 \cdot 10^{-14})$	$(1.46 \cdot 10^{-4}, 6.29 \cdot 10^{-14})$	$5.34 \cdot 10^{-9}$

49 / 78

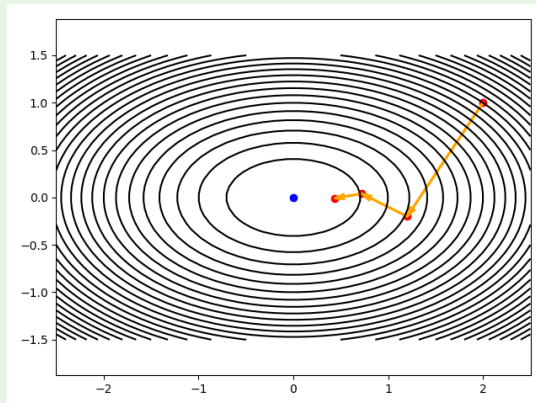
Descente du gradient - deux variables



Point initial

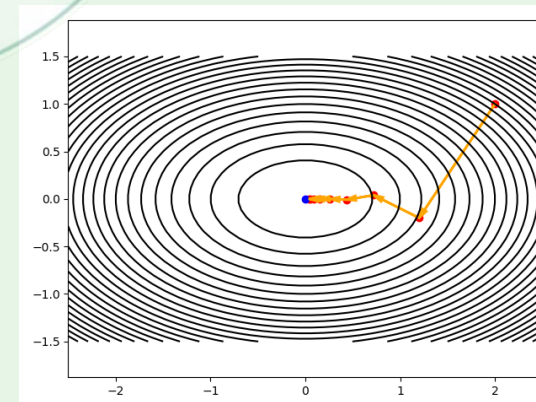
50 / 78

Descente du gradient - deux variables



Trois itérations

Descente du gradient - deux variables



Plus d'itérations

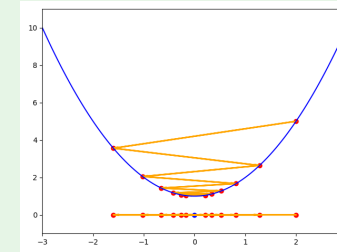


Descente du gradient - Choix du paramètre η

- Le choix du paramètre η est important.
- Le **dilemme** caché derrière ce paramètre est qu'il permet une convergence rapide pour un pas élevé, mais de petites valeurs assurent plus de stabilité à l'algorithme. Un pas trop grand peut empêcher l'algorithme de converger.
- Reprenons la fonction f définie par $f(x) = x^2 + 1$ et testons différentes valeurs du pas η (avec toujours $a_0 = 2$).

Descente du gradient - Choix du paramètre η

- Le choix du paramètre η est important.
- Une **tres grande** valeur pour η risque d'oscillation !!



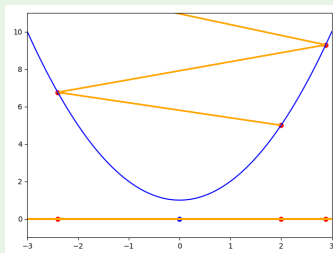
- Pour $\eta = 0.9$, la suite (a_k) tend bien vers $a_{\min} = 0$. Les ordonnées sont bien décroissantes mais comme η est trop grand, la suite des points oscille de part et d'autre du minimum.

53 / 78

54 / 78

Descente du gradient - Choix du paramètre η

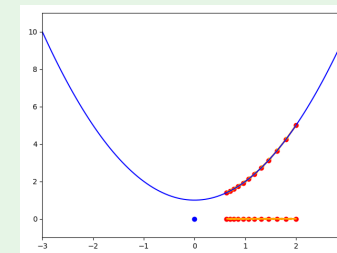
- Le choix du paramètre η est important.
- Une **tres grande** valeur pour η risque d'oscillation !!



- Pour $\eta = 1.1$, la suite (a_k) diverge. Les ordonnées augmentent, la suite des points oscille et s'échappe. Cette valeur de η ne donne pas de convergence vers un minimum.

Descente du gradient - Choix du paramètre η

- Le choix du paramètre η est important.
- Une **tres petite** valeur pour η augmente la quantité du calcul.



- Pour $\eta = 0.05$, la suite (a_k) tend bien vers $a_{\min}$ mais, comme η est trop petit, il faudrait beaucoup d'itérations pour arriver à une approximation raisonnable.



Descente du gradient - Choix du point initial a_0

Le choix du point de départ est également important

- En particulier, lorsqu'il existe plusieurs minimums locaux.
- Soit la fonction f définie par :

$$f(a) = a^4 - 5a^2 + a + 10.$$

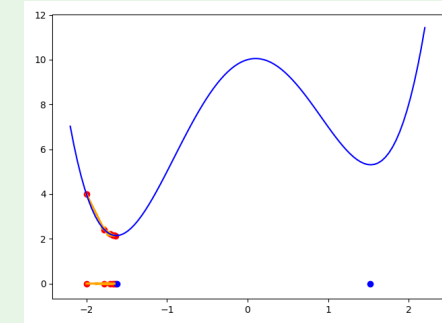
Cette fonction admet deux minimums locaux.

- La suite (a_k) de la descente de gradient converge vers l'un de ces deux minimums selon le choix du point initial a_0 (ici $\eta = 0.02$).

57 / 78

Descente du gradient - Choix du point initial a_0

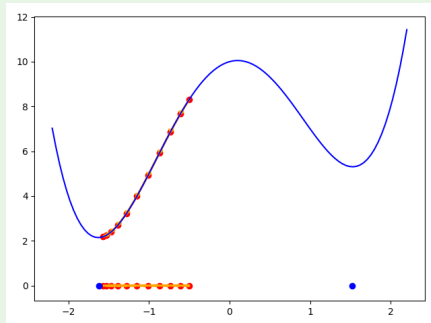
- Le choix du point de départ est également important
- En particulier, lorsqu'il existe plusieurs minimums locaux.
- $a_0 = -2$



58 / 78

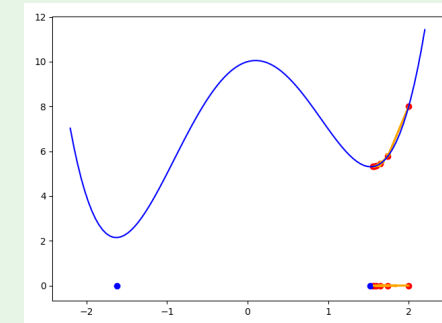
Descente du gradient - Choix du point initial a_0

- Le choix du point de départ est également important
- En particulier, lorsqu'il existe plusieurs minimums locaux.
- $a_0 = -0.5$



Descente du gradient - Choix du point initial a_0

- Le choix du point de départ est également important
- En particulier, lorsqu'il existe plusieurs minimums locaux.
- $a_0 = 2$





Descente du gradient - Approche

Maintenant que nous avons compris le principe de la méthode du gradient pour trouver un minimum local d'une fonction, revenons à la problématique de l'algorithme:

- Nous cherchons à mettre à jours les poids afin de minimiser la fonction de coût $J(w)$.
- La méthode du gradient nous dit pour trouver le point w qui minimise $J(w)$, il faut faire une mise à jour de w à chaque itération en faisant un pas dans la direction opposée du gradient, soit de :

$$\Delta w = -\eta \nabla J(w)$$

- Comme pour les exemples précédents, nous devons donc commencé par calculer $\nabla J(w)$

61 / 78

Descente du gradient - Calcul du $\nabla J(w)$ pour un neurone

Pour un poids w_j donné :

$$\begin{aligned} \frac{\partial J}{\partial w_j} &= \frac{\partial}{\partial w_j} \frac{1}{2} \sum_i (y^i - \hat{y}^i)^2 \\ &= \frac{1}{2} \sum_i \frac{\partial}{\partial w_j} (y^i - \hat{y}^i)^2 \\ &= \frac{1}{2} \sum_i 2 \cdot (y^i - \hat{y}^i) \frac{\partial}{\partial w_j} (y^i - \hat{y}^i) \\ &= \sum_i (y^i - \hat{y}^i) \frac{\partial}{\partial w_j} \left(y^i - \sum_j w_j x_j^i \right) \\ &= \sum_i (y^i - \hat{y}^i) (-x_j^i) \end{aligned}$$

62 / 78

Descente du gradient - Calcul de la mise à jour Δw_j

- Maintenant que nous avons calculé le gradient par rapport aux poids w_j , nous pouvons calculer sa mise à jour.
- La mise à jour est donnée par : $\Delta w_j = -\eta \frac{\partial}{\partial w_j}$

$$\begin{aligned} \Delta w_j &= -\eta \frac{\partial}{\partial w_j} \\ &= -\eta \sum_i (y^i - \hat{y}^i) (-x_j^i) \\ &= \eta \sum_i (y^i - \hat{y}^i) x_j^i \end{aligned}$$

- la nouvelle valeur du w_j est : $w_j = w_j + \Delta w_j$

Descente du gradient - Calcul de la mise à jour Δw_j

- Nous savons calculer le gradient:

$$\nabla J(w) = \frac{\partial J(w)}{\partial w} = \left(\frac{\partial J}{\partial w_0}, \dots, \frac{\partial J}{\partial w_j}, \dots, \frac{\partial J}{\partial w_n} \right)$$

- Voici le vecteur des mises à jour : Δw

$$\Delta w = -\eta \nabla J(w) = \left(-\eta \frac{\partial J}{\partial w_0}, \dots, -\eta \frac{\partial J}{\partial w_j}, \dots, -\eta \frac{\partial J}{\partial w_n} \right)$$

- Finalement, nous pouvons appliquer une mise à jour simultanée de tous poids : $w = w + \Delta w$



Algorithme de la descente du gradient

- 1 **Entrées:** Un ensemble d'entraînement D contenant n exemples (x_i, y_i) ; $x_i \in \mathbb{R}^d$ et $y_i \in \mathbb{R}$
- 2 Initialiser les poids w et le seuil b ;
- 3 **répéter**
 - 4 Calculer le gradient $\nabla J(w) = \frac{\partial J(w)}{\partial w}$
 - 5 Mettre à jour les poids $w = w - \eta \frac{\partial J(w)}{\partial w}$
- 6 **jusqu'à la convergence;**
- 7 **Sortie:** Le poids du perceptron (w, b) ;

```
def descente(X, y):
    w = np.zeros(1 + X.shape[1])

    for i in range(epochs):
        yhat = predict(X)
        w += eta * X.T.dot(y - yhat)

    return w
```

65 / 78

66 / 78

Algorithme d'apprentissage

Le perceptron : descente du gradient stochastique

Comparaison de deux algorithmes

Bien que la règle d'apprentissage ci-dessus semble identique à la règle du perceptron, nous noterons les deux principales différences :

- Ici la sortie $\hat{y}$ la sortie est un nombre réel et non une étiquette de classe comme dans la règle d'apprentissage du perceptron.
- La mise à jour des poids est calculée sur la base de tous les exemples de l'ensemble d'entraînement (off-line) alors que la règle du perceptron est en ligne.

Variantes de la descente de gradient

Il existe trois variantes principales de la descente du gradient :

- **Descentes de gradient de lot (batch)** ou descente classique : dans cette variante, le gradient est calculé sur l'ensemble des données d'entraînement, et les paramètres sont mis à jour après chaque époque.
- **Descente de gradient stochastique** : dans cette variante, le gradient est calculé sur un seul exemple d'entraînement, et les paramètres sont mis à jour après chaque exemple.
- **Descente de gradient de mini-lots** : Dans cette variante, le gradient est calculé sur un petit sous-ensemble des données d'entraînement, et les paramètres sont mis à jour après chaque mini-lot.



Algorithme de la descente du gradient stochastique

- 1 Un ensemble d'entraînement D contenant n exemples (x_i, y_i) ; $x_i \in \mathbb{R}^d$ et $y_i \in \mathbb{R}$
- 2 Initialiser les poids w et le seuil b ;
- 3 **répéter**
 - 4 **pour $i=0$ à n faire**
 - 5 Prendre x^i de X
 - 6 Calculer $\hat{y}^i$ la prediction de x^i
 - 7 Calculer le gradient $\frac{\partial J_i(w)}{\partial w} = (y^i - \hat{y}^i) x^i$
 - 8 Mettre à jour les poids $w = w - \eta (y^i - \hat{y}^i) x^i$
- 9 **jusqu'à La convergence;**
- 10 **Sortie:** Le poids du perceptron (w, b) ;

```
def sgd(X, y):
    w = np.zeros(1 + X.shape[1])

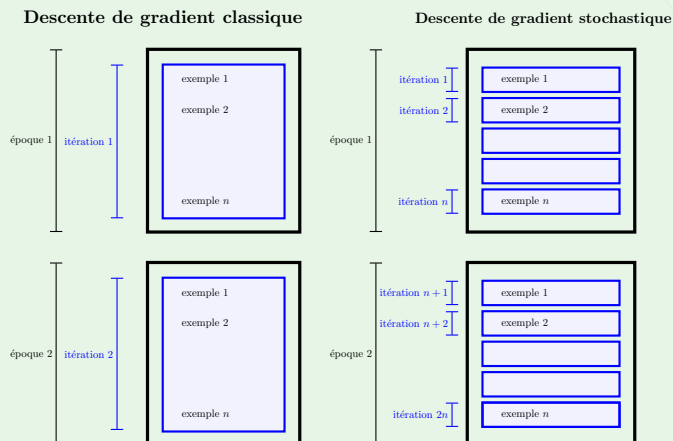
    for i in range(epochs):
        for xi, yi in zip(X, y):
            yhat = predict(xi)
            w += eta * xi.dot (yi - yhat)

    return w
```

69 / 78

70 / 78

Descente classique vs descente stochastique



Les types de fonctions d'erreur

- La fonction d'erreur $J(w)$ utilisée jusqu'ici est la somme des carrées des erreurs:

$$J(w) = \frac{1}{2} \sum_i (y^i - \hat{y}^i)^2$$

- Il existe différentes formules pour calculer l'erreur entre la sortie attendue y_i et la sortie calculée $\hat{y}_i$.



Erreur quadratique moyenne

Erreur quadratique moyenne :

$$J(w) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

- C'est la formule la plus classique (en anglais **minimal squared error** ou **mse**).
- $J(w) \geq 0$ quels que soit les $\hat{y}_i$ et $E = 0$ si et seulement si $y_i = \hat{y}_i$ pour tous les $i = 1, \dots, n$.

73 / 78

Algorithme d'apprentissage

Résumé du vocabulaire avec sa traduction en anglais :

- descente de gradient (classique), **(batch) gradient descent**,
- descente de gradient stochastique, **sgd** pour **stochastic gradient descent**,
- descente de gradient par lots, **mini-batch gradient descent**,
- pas η , **learning rate**,
- erreur quadratique moyenne, **mse** pour **minimal squared error**,
- époque, **epoch**.

Erreur absolue moyenne

- Erreur absolue moyenne:

$$J(w) = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

- C'est une formule plus naturelle, mais moins agréable à manipuler à cause de la valeur absolue.
- Noter que pour ces deux formules, l'erreur globale $J(w)$ est la moyenne d'erreurs locales $J(w_j) = (y_j - \hat{y}_j)^2$ (ou bien $J(w_j) = |y_j - \hat{y}_j|$).
- Les erreurs locales sont indépendantes les unes des autres, ce qui est la base de la descente de gradient stochastique.
- Il existe d'autres formules d'erreur que nous allons les étudier plus loin dans ce cours.

74 / 78

Ce que j'ai appris aujourd'hui

Nous avons vu dans ce cours:

- 1 Les deux types neurones artificiels: le neurone formel ou le neurone MP et le perceptron (appelé aussi neurone tout cout).
- 2 Le neurone MP est limité: traite uniquement les fonctions booléennes, le seuil est fixé manuellement, ne permet pas de favoriser des entrées par rapport aux autres;
- 3 Le perceptron peut apprendre grace aux paramètres et biais, peut prendre des entrées réelles.
- 4 Un seul neurone ne peut pas traiter les données **non linéairement séparables**.
- 5 Principe de l'algorithme d'apprentissage d'un neurone par correction d'erreurs, règle du perceptron.

Ce que j'ai appris aujourd'hui



1 Principe de la méthode de la descente du gradient pour la recherche d'un minimum.

2 Comment appliquer la descente de gradient pour minimiser la fonction de perte (coût) pour un neurone !!

3 Quel est le taux d'apprentissage, pourquoi il est important et son impact sur les résultats.

4 importance de la valeur du point de départ (valeur initiale) et son impact la vitesse de la convergence de l'algorithme.

5 Comment écrire le code pour la descente de gradient, **que je doit impérativement implémenter en TP**

6 j'ai saisi la différence entre les différentes variantes de la descente du gradient et vu quelques fonctions d'erreur.

Cours suivant

Nous allons étudier :

- **Les fonctions non linéairement séparables** :
ou comment combiner plusieurs neurones pour prendre en charge les fonctions non linéairement séparables.
- **Multi-couches de neurones** :
présentation et mise en équation
- **Fonctions d'activation** :
étudier les différents types des fonctions utilisées

